



Projet de développement PGBD Rapport final

Table des matières

Table des matières.....	1
Définition des besoins.....	2
Identification des éléments clés.....	3
Acteurs et intervenants externes.....	3
Les fonctionnalités principales.....	4
Les principales entités de données.....	5
Modèle entité-association.....	6
Schéma physique de la base de données.....	7
Prototypes des écrans.....	8
Schéma d'architecture.....	12
Structure de la solution .NET.....	13
Script SQL de création de la base de données.....	14
Présentation de l'application.....	29
Propriétés.....	30
Dossiers.....	31
Personnes.....	32
Conclusion.....	33

Définition des besoins

EasyImmo est une application de gestion immobilière destinée exclusivement aux agents immobiliers. Une fois connectés, les agents accèdent à un tableau de bord centralisé depuis lequel ils peuvent piloter l'ensemble de leurs activités quotidiennes. Au cœur de l'application se trouve la gestion des propriétés. Chaque bien immobilier peut être ajouté, modifié ou supprimé par l'agent. Une propriété est illustrée par des photos et dispose d'un historique de statuts complet permettant de retracer son évolution dans le temps, avec les dates précises de chaque changement. À chaque propriété peuvent être rattachées une ou plusieurs annonces, chacune possédant également son propre statut et son historique, offrant ainsi une visibilité totale sur le cycle de vie commercial du bien.

Chaque annonce peut générer plusieurs dossiers clients. Un dossier représente le suivi d'un candidat, qu'il soit acheteur ou locataire, et progresse à travers différentes étapes clairement définies telles que la demande de renseignements, la visite, le dépôt d'offres ou encore la signature du contrat. Chaque étape est tracée avec sa date et ses notes associées. Des fichiers au peuvent être joints à un dossier afin de centraliser les documents importants comme les justificatifs, les offres ou les contrats. Tout comme les propriétés et les annonces, chaque dossier dispose de son propre statut.

Les agents peuvent également gérer les personnes gravitant autour de leurs biens, qu'il s'agisse de propriétaires, de locataires ou de simples visiteurs. Les coordonnées et informations de chaque personne sont accessibles rapidement et peuvent être reliées aux propriétés, annonces ou dossiers concernés.

Enfin, l'application intègre un système d'événements permettant d'enregistrer toute action significative liée à une propriété ou à un dossier, avec ou sans personne impliquée. Lorsqu'un événement correspond à un rendez-vous ou à une visite, il apparaît automatiquement dans le calendrier intégré de l'application, donnant ainsi aux agents une vue d'ensemble claire et organisée sur leurs prochaines échéances.

Identification des éléments clés

Acteurs et intervenants externes

L'agent immobilier est l'acteur principal de l'application. C'est le seul utilisateur ayant accès au panel et il est responsable de la gestion complète des propriétés, des annonces, des dossiers et des événements.

Le propriétaire est la personne physique ou morale qui confie son bien à l'agence. Il est référencé dans l'application mais n'y a pas accès directement. Il intervient comme contact lié à une propriété.

Le candidat locataire ou acheteur est la personne qui manifeste un intérêt pour un bien mis en annonce. Il est suivi à travers un dossier client et ses informations personnelles sont enregistrées par l'agent dans l'application.

Le locataire est le candidat dont le dossier a abouti à la signature d'un contrat de location. Il reste référencé dans l'application comme intervenant actif sur la propriété concernée.

Le notaire intervient en tant qu'acteur externe dans le cadre des transactions de vente. Bien qu'il n'ait pas accès à l'application, ses interventions peuvent être tracées sous forme d'étapes ou d'événements dans un dossier.

Le visiteur est une personne ayant effectué une visite d'un bien sans nécessairement avoir ouvert un dossier complet. Il peut être enregistré dans l'application pour garder une trace de son passage.

Les fonctionnalités principales

La gestion des propriétés constitue le cœur de l'application. L'agent peut ajouter, modifier ou supprimer un bien immobilier, y associer des photos, suivre son statut en temps réel et consulter l'historique complet de ses changements d'état.

La gestion des annonces permet à l'agent de créer et de suivre une ou plusieurs annonces rattachées à une propriété. Chaque annonce possède son propre statut et son historique, offrant une traçabilité complète sur la mise en marché du bien.

La gestion des dossiers clients permet de suivre chaque candidat acheteur ou locataire à travers un dossier structuré. Ce dossier progresse par étapes, chacune étant datée et annotée. Des fichiers PDF peuvent y être joints et un statut évolutif permet de connaître à tout moment l'avancement du dossier.

La gestion des personnes permet d'enregistrer et de consulter les coordonnées de tous les intervenants gravitant autour des biens, qu'il s'agisse de propriétaires, de locataires, de candidats ou de visiteurs.

La gestion des événements permet d'enregistrer toute action ou étape significative liée à une propriété ou à un dossier. Lorsqu'un événement correspond à une visite ou un rendez-vous, il est automatiquement intégré au calendrier de l'application.

La gestion du calendrier offre à l'agent une vue d'ensemble claire et chronologique de tous ses rendez-vous et visites à venir, générés automatiquement depuis les événements enregistrés dans l'application.

La gestion des accès garantit que seuls les agents immobiliers authentifiés peuvent accéder à l'application, assurant ainsi la confidentialité et la sécurité des données.

Les principales entités de données

Entité	Description
Property	Représente un bien immobilier
PropertyDetail	Regroupe les caractéristiques techniques et détaillées d'une propriété
Person	Représente tout intervenant externe
Agent	Représente un agent immobilier
Listing	Représente une annonce créée pour une propriété
Folder	Représente le dossier d'un candidat acheteur ou locataire
FolderStep	Représente une étape franchie dans l'avancement d'un dossier
Event	Représente un événement lié à une propriété ou un dossier



Modèle entité-association

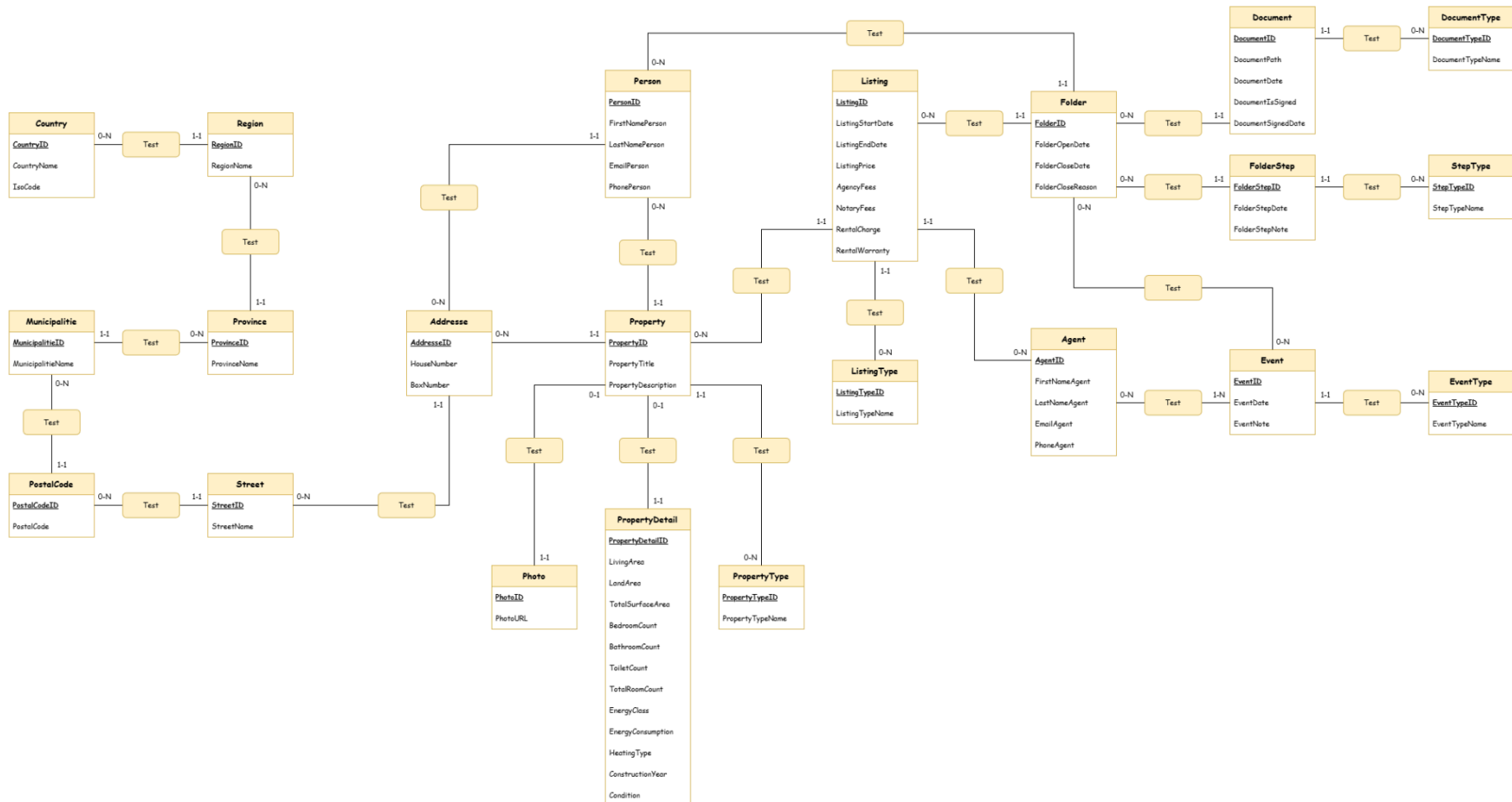
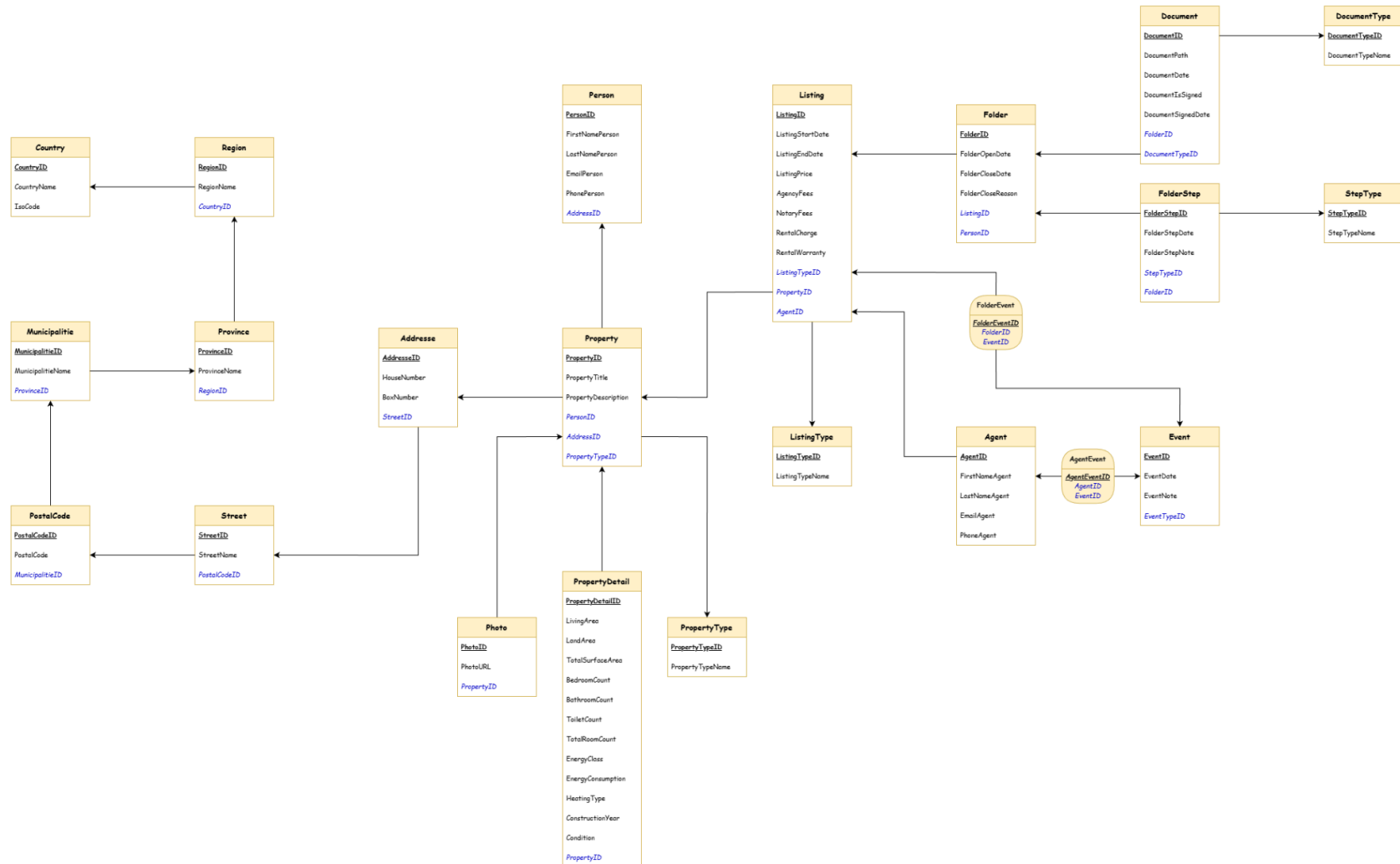




Schéma physique de la base de données



Prototypes des écrans

Dashboard

EasyImmo

- Dashboard
- Propriétés
- Dossiers
- Personnes
- Calendrier
- Boite mail

Se déconnecter

Phrase d'accroche

36 Propriétés

36 Personnes

36 Dossiers

36 Rendez-vous

Dernières propriétés

Derniers dossiers

Propriétés

EasyImmo

- Dashboard
- Propriétés
- Dossiers
- Personnes
- Calendrier
- Boite mail

Se déconnecter

Propriétés

Description

Type
Titre de la propriété
Informations
Adresse

V M S

Type
Titre de la propriété
Informations
Adresse

V M S

Type
Titre de la propriété
Informations
Adresse

V M S

Type
Titre de la propriété
Informations
Adresse

V M S

Type
Titre de la propriété
Informations
Adresse

V M S

Type
Titre de la propriété
Informations
Adresse

V M S

Détails propriété

EasyImmo

Dashboard

Propriétés

Dossiers

Personnes

Calendrier

Boite mail

Se déconnecter

Photo

Type de propriété
Titre
Adresse
Description

Caractéristiques

Energie

Propriétaire

Annonce

Dossiers

EasyImmo

Dashboard

Propriétés

Dossiers

Personnes

Calendrier

Boite mail

Se déconnecter

Dossiers

Description

Dossier numéro 1
Nom Prénom

Statut

Dossier numéro 1
Nom Prénom

Statut

Dossier numéro 1
Nom Prénom

Statut

Dossier numéro 1
Nom Prénom

Statut

Détails dossiers

EasyImmo

Dashboard

Propriétés

Dossiers

Personnes

Calendrier

Boite mail

Se déconnecter

Dossier numéro
Statut : Ouvert

Modifier le statut

Ajouter une étape

Informations

Annonce

Etapes du dossier

Personnes

EasyImmo

Dashboard

Propriétés

Dossiers

Personnes

Calendrier

Boite mail

Se déconnecter

Personnes

Description

Q Search

Ajouter une personne

Nom Prénom
Email

VMS

Nom Prénom
Email

VMS

Nom Prénom
Email

VMS

Détails personne

EasyImmo

Dashboard

Propriétés

Dossiers

Personnes

Calendrier

Boite mail

Se déconnecter

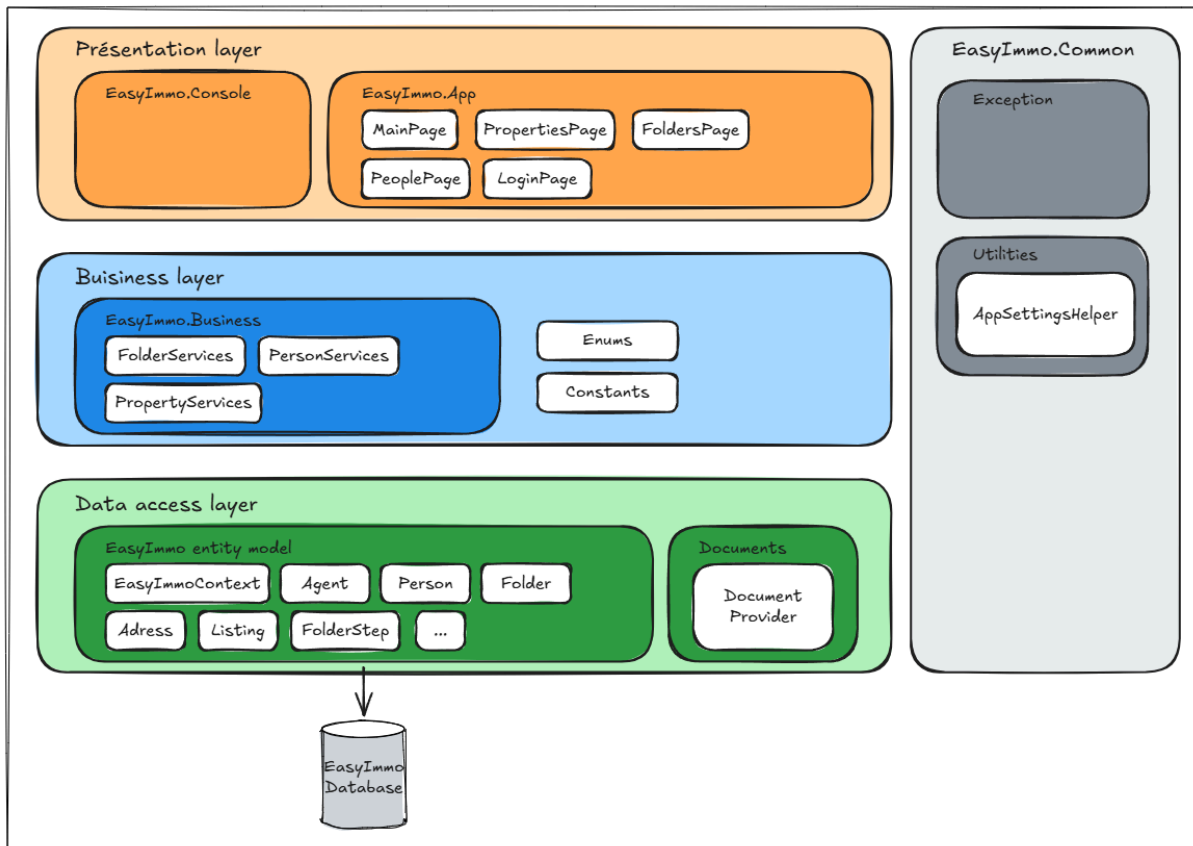
Nom Prénom
Email

Informations

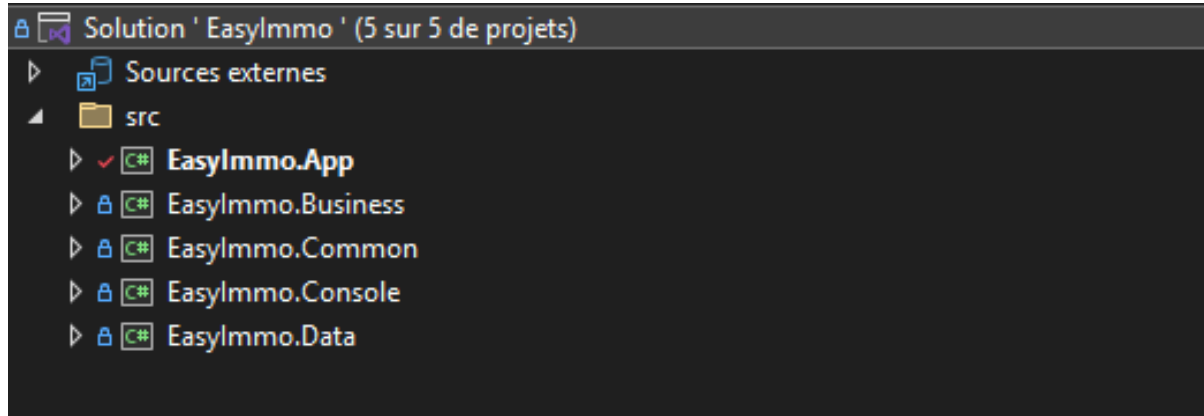
Propriétés

Dossiers

Schéma d'architecture



Structure de la solution .NET



EasyImmo.App : Couche de présentation. Contient l'interface utilisateur développée en .NET MAUI. C'est le point d'entrée de l'application. Elle contient les pages, les vues et les ViewModels.

EasyImmo.Business : Couche métier. Contient la logique applicative, les règles métier et les services. Elle fait le lien entre la couche de présentation et la couche d'accès aux données.

EasyImmo.Common : Contient les éléments partagés entre les autres couches : constantes, enums, utilitaires, etc.

EasyImmo.Console : Projet console permettant de tester rapidement la logique métier et l'accès aux données sans passer par l'interface graphique.

EasyImmo.Data : Couche d'accès aux données. Contient les entités, le contexte Entity Framework, les migrations et les accès à la base de données.

Script SQL de création de la base de données

```
/****** Object: Database [EasyImmo] *****/
CREATE DATABASE [EasyImmo]
GO

USE [EasyImmo]
GO

/****** Object: Table [dbo].[Address] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Address](
    [AddressID] [int] IDENTITY(1,1) NOT NULL,
    [HouseNumber] [int] NOT NULL,
    [BoxNumber] [nvarchar](20) NULL,
    [StreetID] [int] NOT NULL,
    PRIMARY KEY CLUSTERED
    (
        [AddressID] ASC
    )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
    IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
    ON [PRIMARY]
) ON [PRIMARY]
GO

/****** Object: Table [dbo].[Agent] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Agent](
    [AgentID] [int] IDENTITY(1,1) NOT NULL,
    [FirstNameAgent] [nvarchar](50) NOT NULL,
    [LastNameAgent] [nvarchar](50) NOT NULL,
    [EmailAgent] [nvarchar](100) NOT NULL,
    [PhoneAgent] [nvarchar](20) NOT NULL,
```

```
[PasswordAgent] [nvarchar](255) NOT NULL,
PRIMARY KEY CLUSTERED
(
    [AgentID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[AgentEvent] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[AgentEvent](
    [AgentEventID] [int] IDENTITY(1,1) NOT NULL,
    [AgentID] [int] NOT NULL,
    [EventID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [AgentEventID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Country] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Country](
    [CountryID] [int] IDENTITY(1,1) NOT NULL,
    [CountryName] [nvarchar](100) NOT NULL,
    [IsoCode] [nvarchar](10) NOT NULL,
PRIMARY KEY CLUSTERED
(
    [CountryID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
```



```
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Document] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Document](
    [DocumentID] [int] IDENTITY(1,1) NOT NULL,
    [DocumentPath] [nvarchar](255) NOT NULL,
    [DocumentDate] [datetime] NOT NULL,
    [DocumentIsSigned] [bit] NOT NULL,
    [DocumentSignatureDate] [datetime] NULL,
    [DocumentTypeID] [int] NOT NULL,
    [FolderID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [DocumentID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[DocumentType] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[DocumentType](
    [DocumentTypeID] [int] IDENTITY(1,1) NOT NULL,
    [DocumentTypeName] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
(
    [DocumentTypeID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
```

```
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Event] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Event](
    [EventID] [int] IDENTITY(1,1) NOT NULL,
    [EventDate] [datetime] NOT NULL,
    [EventNote] [nvarchar](255) NULL,
    [EventTypeID] [int] NOT NULL,
    PRIMARY KEY CLUSTERED
(
    [EventID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[EventType] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[EventType](
    [EventTypeID] [int] IDENTITY(1,1) NOT NULL,
    [EventTypeName] [nvarchar](50) NOT NULL,
    PRIMARY KEY CLUSTERED
(
    [EventTypeID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Folder] *****/
SET ANSI_NULLS ON
```

```
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Folder](
    [FolderID] [int] IDENTITY(1,1) NOT NULL,
    [FolderOpenDate] [datetime] NOT NULL,
    [FolderCloseDate] [datetime] NULL,
    [FolderCloseReason] [nvarchar](255) NULL,
    [ListingID] [int] NOT NULL,
    [PersonID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [FolderID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[FolderEvent] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[FolderEvent](
    [FolderEventID] [int] IDENTITY(1,1) NOT NULL,
    [FolderID] [int] NOT NULL,
    [EventID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [FolderEventID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[FolderStep] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
```

```
GO
CREATE TABLE [dbo].[FolderStep](
    [FolderStepID] [int] IDENTITY(1,1) NOT NULL,
    [FolderStepDate] [datetime] NOT NULL,
    [FolderStepNote] [nvarchar](255) NULL,
    [FolderID] [int] NOT NULL,
    [StepTypeID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [FolderStepID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Listing] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Listing](
    [ListingID] [int] IDENTITY(1,1) NOT NULL,
    [ListingStartDate] [datetime] NOT NULL,
    [ListingEndDate] [datetime] NULL,
    [ListingPrice] [float] NOT NULL,
    [AgencyFees] [float] NULL,
    [NotaryFees] [float] NULL,
    [RentalCharge] [float] NULL,
    [RentalWarranty] [float] NULL,
    [ListingTypeID] [int] NOT NULL,
    [PropertyID] [int] NOT NULL,
    [AgentID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [ListingID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO
```

```
/****** Object: Table [dbo].[ListingType] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[ListingType](
    [ListingTypeID] [int] IDENTITY(1,1) NOT NULL,
    [ListingTypeName] [nvarchar](50) NOT NULL,
    PRIMARY KEY CLUSTERED
(
    [ListingTypeID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/****** Object: Table [dbo].[Municipality] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Municipality](
    [MunicipalityID] [int] IDENTITY(1,1) NOT NULL,
    [MunicipalityName] [nvarchar](100) NOT NULL,
    [ProvinceID] [int] NOT NULL,
    PRIMARY KEY CLUSTERED
(
    [MunicipalityID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/****** Object: Table [dbo].[Person] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
```

```
CREATE TABLE [dbo].[Person](
    [PersonID] [int] IDENTITY(1,1) NOT NULL,
    [FirstNamePerson] [nvarchar](50) NOT NULL,
    [LastNamePerson] [nvarchar](50) NOT NULL,
    [EmailPerson] [nvarchar](100) NOT NULL,
    [PhonePerson] [nvarchar](20) NOT NULL,
    [AddressID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [PersonID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Photo] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Photo](
    [PhotoID] [int] IDENTITY(1,1) NOT NULL,
    [PhotoURL] [nvarchar](255) NOT NULL,
    [PropertyID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [PhotoID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[PostalCode] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[PostalCode](
    [PostalCodeID] [int] IDENTITY(1,1) NOT NULL,
```

```
[PostalCode] [nvarchar](20) NOT NULL,  
[MunicipalityID] [int] NOT NULL,  
PRIMARY KEY CLUSTERED  
(  
    [PostalCodeID] ASC  
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,  
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)  
ON [PRIMARY]  
) ON [PRIMARY]  
GO  
  
/***** Object: Table [dbo].[Property] *****/  
SET ANSI_NULLS ON  
GO  
SET QUOTED_IDENTIFIER ON  
GO  
CREATE TABLE [dbo].[Property](  
    [PropertyID] [int] IDENTITY(1,1) NOT NULL,  
    [PropertyTitle] [nvarchar](255) NOT NULL,  
    [PropertyDescription] [nvarchar](max) NULL,  
    [PersonID] [int] NOT NULL,  
    [PropertyTypeID] [int] NOT NULL,  
    [AddressID] [int] NOT NULL,  
PRIMARY KEY CLUSTERED  
(  
    [PropertyID] ASC  
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,  
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)  
ON [PRIMARY]  
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]  
GO  
  
/***** Object: Table [dbo].[PropertyDetail] *****/  
SET ANSI_NULLS ON  
GO  
SET QUOTED_IDENTIFIER ON  
GO  
CREATE TABLE [dbo].[PropertyDetail](  
    [PropertyDetailID] [int] IDENTITY(1,1) NOT NULL,  
    [LivingArea] [float] NULL,  
    [LandArea] [float] NULL,
```

```
[TotalSurfaceArea] [float] NULL,  
[BedroomCount] [int] NULL,  
[BathroomCount] [int] NULL,  
[ToiletCount] [int] NULL,  
[TotalRoomCount] [int] NULL,  
[EnergyClass] [nvarchar](20) NULL,  
[EnergyConsumption] [float] NULL,  
[HeatingType] [nvarchar](50) NULL,  
[ConstructionYear] [int] NULL,  
[Condition] [nvarchar](50) NULL,  
[PropertyID] [int] NOT NULL,  
PRIMARY KEY CLUSTERED  
(  
    [PropertyDetailID] ASC  
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,  
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)  
ON [PRIMARY]  
) ON [PRIMARY]  
GO  
  
/***** Object: Table [dbo].[PropertyType] *****/  
SET ANSI_NULLS ON  
GO  
SET QUOTED_IDENTIFIER ON  
GO  
CREATE TABLE [dbo].[PropertyType](  
    [PropertyTypeID] [int] IDENTITY(1,1) NOT NULL,  
    [PropertyTypeName] [nvarchar](50) NOT NULL,  
    PRIMARY KEY CLUSTERED  
(  
        [PropertyTypeID] ASC  
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,  
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)  
ON [PRIMARY]  
) ON [PRIMARY]  
GO  
  
/***** Object: Table [dbo].[Province] *****/  
SET ANSI_NULLS ON  
GO  
SET QUOTED_IDENTIFIER ON
```



```
GO
CREATE TABLE [dbo].[Province](
    [ProvinceID] [int] IDENTITY(1,1) NOT NULL,
    [ProvinceName] [nvarchar](100) NOT NULL,
    [RegionID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [ProvinceID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Region] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Region](
    [RegionID] [int] IDENTITY(1,1) NOT NULL,
    [RegionName] [nvarchar](100) NOT NULL,
    [CountryID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [RegionID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[StepType] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[StepType](
    [StepTypeID] [int] IDENTITY(1,1) NOT NULL,
    [StepTypeName] [nvarchar](50) NOT NULL,
PRIMARY KEY CLUSTERED
```

```
(
    [StepTypeID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Object: Table [dbo].[Street] *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Street](
    [StreetID] [int] IDENTITY(1,1) NOT NULL,
    [StreetName] [nvarchar](100) NOT NULL,
    [PostalCodeID] [int] NOT NULL,
PRIMARY KEY CLUSTERED
(
    [StreetID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
) ON [PRIMARY]
GO

/***** Foreign Keys *****/
ALTER TABLE [dbo].[Address] WITH NOCHECK ADD FOREIGN
KEY([StreetID])
REFERENCES [dbo].[Street] ([StreetID])
GO
ALTER TABLE [dbo].[AgentEvent] WITH NOCHECK ADD FOREIGN
KEY([AgentID])
REFERENCES [dbo].[Agent] ([AgentID])
GO
ALTER TABLE [dbo].[AgentEvent] WITH NOCHECK ADD FOREIGN
KEY([EventID])
REFERENCES [dbo].[Event] ([EventID])
GO
ALTER TABLE [dbo].[Document] WITH NOCHECK ADD FOREIGN
KEY([DocumentTypeID])
```

```
REFERENCES [dbo].[DocumentType] ([DocumentTypeID])
GO
ALTER TABLE [dbo].[Document] WITH NOCHECK ADD FOREIGN
KEY([FolderID])
REFERENCES [dbo].[Folder] ([FolderID])
GO
ALTER TABLE [dbo].[Event] WITH NOCHECK ADD FOREIGN
KEY([EventTypeID])
REFERENCES [dbo].[EventType] ([EventTypeID])
GO
ALTER TABLE [dbo].[Folder] WITH NOCHECK ADD FOREIGN
KEY([ListingID])
REFERENCES [dbo].[Listing] ([ListingID])
GO
ALTER TABLE [dbo].[Folder] WITH NOCHECK ADD FOREIGN
KEY([PersonID])
REFERENCES [dbo].[Person] ([PersonID])
GO
ALTER TABLE [dbo].[FolderEvent] WITH NOCHECK ADD FOREIGN
KEY([EventID])
REFERENCES [dbo].[Event] ([EventID])
GO
ALTER TABLE [dbo].[FolderEvent] WITH NOCHECK ADD FOREIGN
KEY([FolderID])
REFERENCES [dbo].[Folder] ([FolderID])
GO
ALTER TABLE [dbo].[FolderStep] WITH NOCHECK ADD FOREIGN
KEY([FolderID])
REFERENCES [dbo].[Folder] ([FolderID])
GO
ALTER TABLE [dbo].[FolderStep] WITH NOCHECK ADD FOREIGN
KEY([StepTypeID])
REFERENCES [dbo].[StepType] ([StepTypeID])
GO
ALTER TABLE [dbo].[Listing] WITH NOCHECK ADD FOREIGN
KEY([AgentID])
REFERENCES [dbo].[Agent] ([AgentID])
GO
ALTER TABLE [dbo].[Listing] WITH NOCHECK ADD FOREIGN
KEY([ListingTypeID])
REFERENCES [dbo].[ListingType] ([ListingTypeID])
```

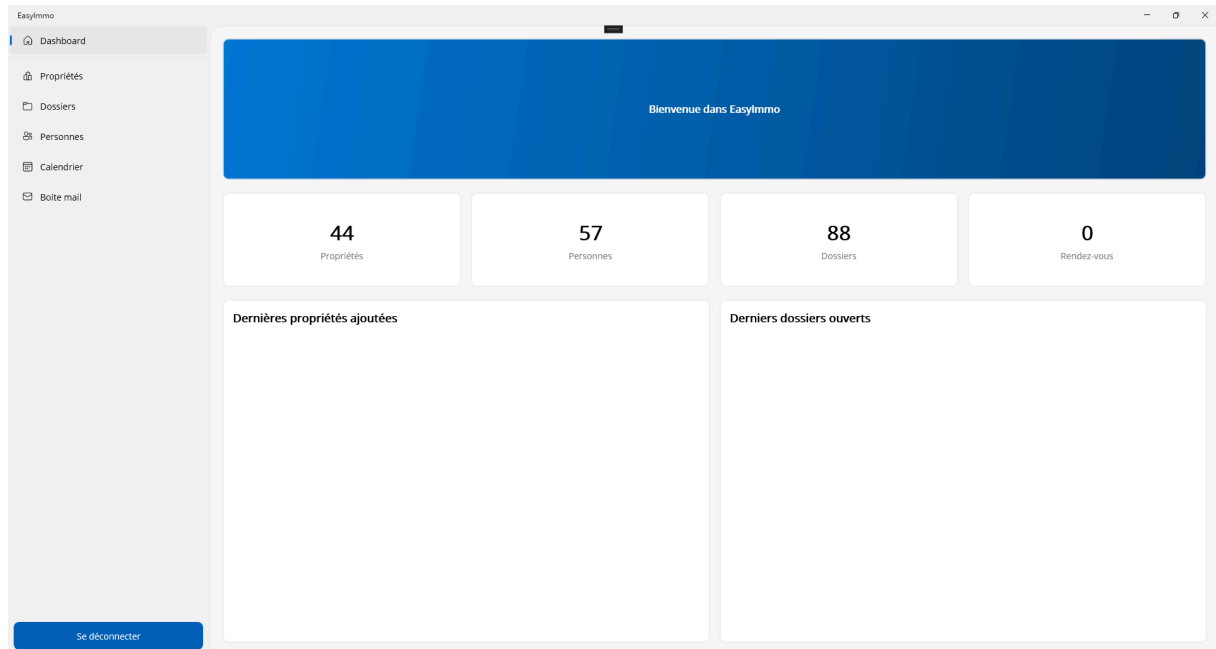
```
GO
ALTER TABLE [dbo].[Listing] WITH NOCHECK ADD FOREIGN
KEY([PropertyID])
REFERENCES [dbo].[Property] ([PropertyID])
GO
ALTER TABLE [dbo].[Municipality] WITH NOCHECK ADD FOREIGN
KEY([ProvinceID])
REFERENCES [dbo].[Province] ([ProvinceID])
GO
ALTER TABLE [dbo].[Person] WITH NOCHECK ADD FOREIGN
KEY([AddressID])
REFERENCES [dbo].[Address] ([AddressID])
GO
ALTER TABLE [dbo].[Photo] WITH NOCHECK ADD FOREIGN
KEY([PropertyID])
REFERENCES [dbo].[Property] ([PropertyID])
GO
ALTER TABLE [dbo].[PostalCode] WITH NOCHECK ADD FOREIGN
KEY([MunicipalityID])
REFERENCES [dbo].[Municipality] ([MunicipalityID])
GO
ALTER TABLE [dbo].[Property] WITH NOCHECK ADD FOREIGN
KEY([AddressID])
REFERENCES [dbo].[Address] ([AddressID])
GO
ALTER TABLE [dbo].[Property] WITH NOCHECK ADD FOREIGN
KEY([PersonID])
REFERENCES [dbo].[Person] ([PersonID])
GO
ALTER TABLE [dbo].[Property] WITH NOCHECK ADD FOREIGN
KEY([PropertyTypeID])
REFERENCES [dbo].[PropertyType] ([PropertyTypeID])
GO
ALTER TABLE [dbo].[PropertyDetail] WITH NOCHECK ADD FOREIGN
KEY([PropertyID])
REFERENCES [dbo].[Property] ([PropertyID])
GO
ALTER TABLE [dbo].[Province] WITH NOCHECK ADD FOREIGN
KEY([RegionID])
REFERENCES [dbo].[Region] ([RegionID])
GO
```



```
ALTER TABLE [dbo].[Region] WITH NOCHECK ADD FOREIGN  
KEY([CountryID])  
REFERENCES [dbo].[Country] ([CountryID])  
GO  
ALTER TABLE [dbo].[Street] WITH NOCHECK ADD FOREIGN  
KEY([PostalCodeID])  
REFERENCES [dbo].[PostalCode] ([PostalCodeID])  
GO
```

Présentation de l'application

Dashboard

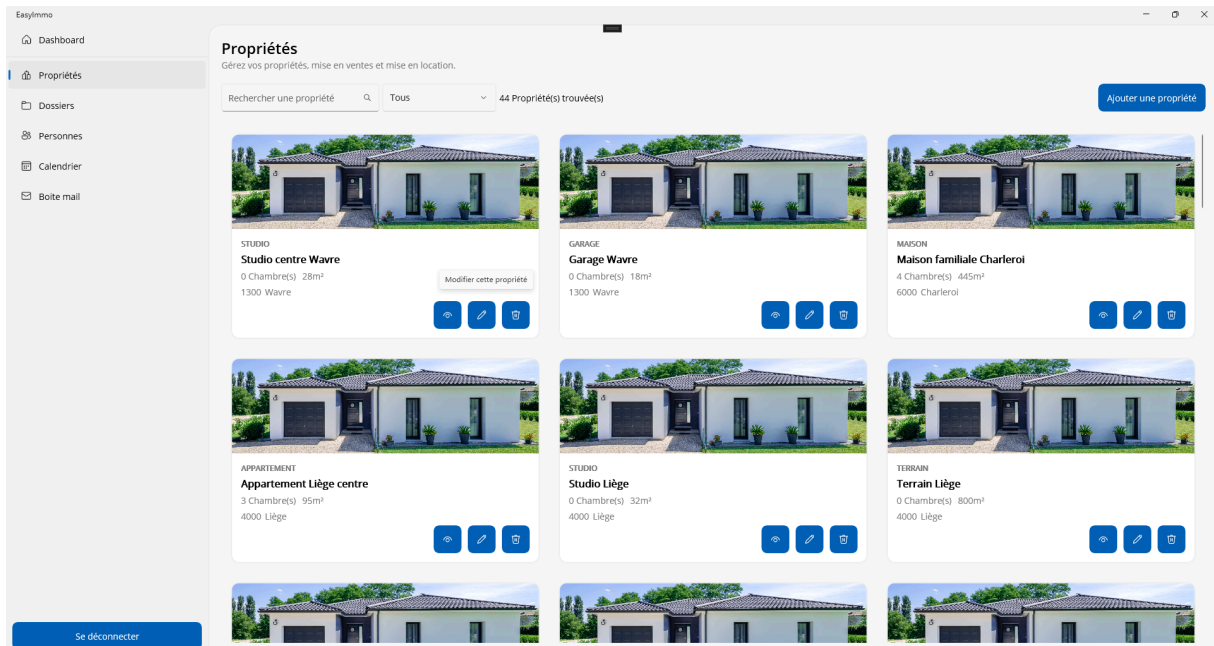


La page d'accueil de EasyImmo est un dashboard qui donne une vue d'ensemble rapide de l'application.

Quatre cartes affichent les statistiques globales en temps réel :

- Le nombre de Propriétés enregistrées
- Le nombre de Personnes dans le système
- Le nombre de Dossiers ouverts
- Le nombre de Rendez-vous planifiés

Propriétés



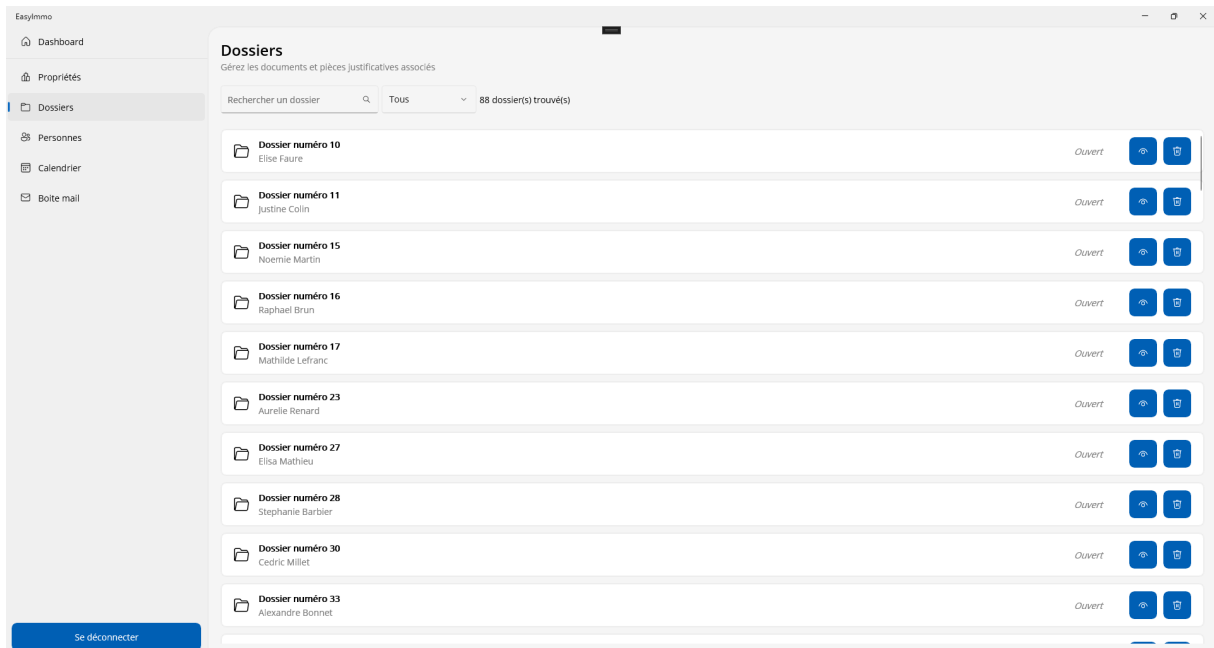
En haut de la page on retrouve une barre d'action composée de :

- Une barre de recherche pour filtrer rapidement une propriété par son nom
- Un menu déroulant permettant de filtrer par type de bien (Tous, Maison, Appartement, Studio, Terrain, Garage, etc.)
- Un compteur indiquant le nombre de propriétés trouvées
- Un bouton "Ajouter une propriété" pour créer une propriété

Les propriétés sont affichées sous forme de grille de cartes. Chaque carte affiche :

- Une photo du bien
- Le type de bien (Studio, Maison, Appartement...)
- Le titre de la propriété
- Le nombre de chambres et la superficie
- La localité et le code postal
- Trois boutons d'action : voir, modifier et supprimer

Dossiers



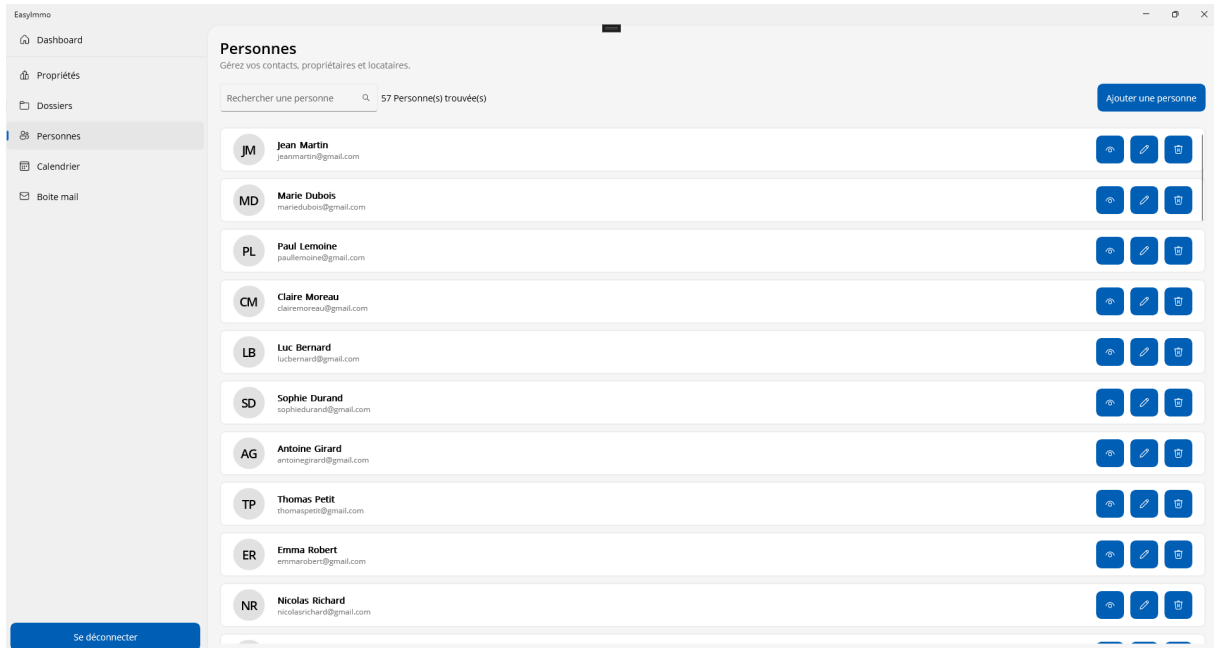
En haut de la page on retrouve la même logique que pour les propriétés avec :

- Une barre de recherche pour trouver rapidement un dossier
- Un menu déroulant pour filtrer par statut (Tous, Ouvert, Fermé...)
- Un compteur affichant le nombre de dossiers trouvés

Contrairement aux propriétés affichées en grille les dossiers sont présentés sous forme de liste. Chaque ligne affiche :

- Une icône de dossier pour identifier visuellement l'élément
- Le numéro du dossier
- Le nom du client associé au dossier
- Le statut du dossier affiché en italique à droite (ex: Ouvert)
- Deux boutons d'action : voir et supprimer le dossier

Personnes



En haut de la page on retrouve :

- Une barre de recherche pour filtrer rapidement une personne par son nom
- Un compteur indiquant le nombre de personnes trouvées (ici 57 personnes)
- Un bouton "Ajouter une personne" pour créer un nouveau contact

Les personnes sont affichées sous forme de liste. Chaque ligne affiche :

- Un avatar circulaire avec les initiales de la personne, permettant une identification visuelle rapide
- Le nom complet de la personne en gras
- Son adresse e-mail
- Trois boutons d'action : voir, modifier et supprimer la personne

Conclusion

Ce projet a été une expérience riche mais honnêtement, je ne peux pas dire que j'en suis entièrement satisfait. J'ai procrastiné je me suis laissé dépasser et au final je n'ai pas réussi à livrer une application utilisable. Est-ce que j'ai été trop ambitieux ? Probablement. Une base de données aussi complète avec autant de fonctionnalités pour un projet seul ça demande une organisation sans faille.

Pourtant, je ne considère pas ce projet comme un échec total. J'ai découvert et appris à utiliser le MVVM, je me suis familiarisé avec le XAML et le développement d'interfaces en .NET MAUI.

Mais la vraie leçon que je retiens c'est ailleurs. Ce projet m'a mis face à mes lacunes en termes de gestion du temps et d'organisation. Me dire que "j'ai encore le temps" alors que les semaines défilent c'est une erreur que je ne peux plus me permettre.

Si je dois reprendre ce projet en deuxième session, je m'y prendrai différemment. Moins d'ambition sur le périmètre, plus de rigueur sur la méthode, et surtout, je m'y mettrai vraiment dès le début.

Annexes

Github

<https://github.com/alexandrerahir/EasyImmo>

Configuration

1. Cloner le repository GitHub
2. Ouvrir SQL Server Management et exécuter le fichier EasyImmo_Script_Data.sql
3. Ouvrir la solution EasyImmo.sln
4. Vérifier la connexion string dans EasyImmo.Data/Entities/[EasyImmoContext.cs](#)
5. Lancer l'application EasyImmo.App